

How can I configure Spring Security?

- If you decide not to use the default configurations provided by Spring Security, you need to write manually a configuration
- You need to write a class annotated with `@Configuration` (do you remember it?) in which you build the beans that you need using `@Bean` annotation
- Typically, you can configure:
 - `UserDetailsManager`
 - `PasswordEncoder`
 - `SecurityFilterChain`
- Let's have a look at a specific object in the `SecurityFilterChain`

42

HTTPSecurity

- `HttpSecurity` is a Spring Security class that allows us to configure:
 - The authentication method (e.g. `HTTBasic`, form login and so on)
 - URL protection
 - Session management
 - Filters
 - CSRF protection
 - Much more!
- A `HttpSecurity` object is a **fluent constructor** of the security configuration
- The method `build()` builds the `SecurityFilterChain` and registers it in the context

43

What Spring Security Does by Default

- By default both form login and HTTP Basic are enabled
- The actual behavior depends on the HTTP request, that is on the headers of the request:

Accept: text/html  Form login

Accept: application/json,
application/xml  HTTP Basic

Authorization: Basic...

The header Accept indicate what the client is expecting

44

Security filter chain with form login

- The UsernamePasswordAuthenticationFilter creates a UsernamePasswordAuthenticationToken, a type of Authentication, by extracting the username and password from the request
- The DaoAuthenticationProvider is used

Please sign in

Are you sure you want to log out?

45

Security filter chain with form login

- Features of the default configuration:

Function	Behaviour
URLs' protection	All requests require authentication (anyRequest().authenticated())
Login	Enabled by default with an auto-generated login page at /login
Logout	Enabled at /logout (it is done via POST, requires CSRF token)
Default user	Username: user Password: randomly generated at runtime (shown in console log)
PasswordEncoder	None or BCryptPasswordEncoder
Session Management	Authentication is stored in HTTP session (JSESSIONID)
CSRF protection	Enabled
UserDetailsService	InMemoryUserDetailsManager with the default user

46

What does the default configuration look like?

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults())
            .logout(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user = User.withUsername("user")
            .password(passwordEncoder().encode("password"))
            .roles("USER")
            .build();
        return new InMemoryUserDetailsManager(user);
    }
}

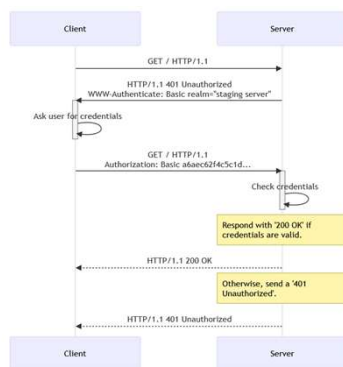
```

When you define your own configuration, Spring disable the default one. You must manage everything yourself!

47

HTTP Basic

- HTTP Basic authentication is the automated default for REST web app. It automatically secures all endpoints with a generated password and the username that the app gets in the header of the HTTP request.



Internet Engineering Task Force (IETF)
Request for Comments: 7617
Obsoletes: 2617
Category: Standards Track
ISSN: 2078-1721

J. Reschke
greenbytes
September 2015

The 'Basic' HTTP Authentication Scheme

Abstract

This document defines the "Basic" Hypertext Transfer Protocol (HTTP) authentication scheme, which transmits credentials as user-id/password pairs, encoded using Base64.

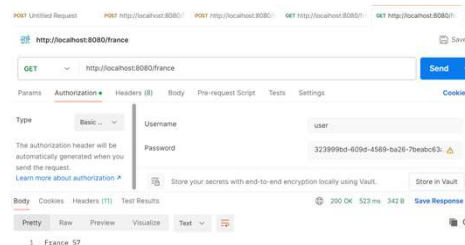
48

Now, is it working?

- Let's check Postman:
 - Start the web app
 - Look at the line in the console:

Using generated security password: 4c30fce1-071f-40c6-b1a0-01ae0881f1ab

- Open Postman:
 - In the Authorization tab choose: Basic Auth, put user in the username field, and copy the password in the password field
 - Send the request



49

Security Filter Chain with HTTP Basic

- HTTP Basic authentication automatically secures all endpoints with a generated password and the username that the app gets in the header of the HTTP request
- The BasicAuthenticationFilter creates a UsernamePasswordAuthenticationToken, a type of Authentication by extracting the username and password from the request
- The DaoAuthenticationProvider is used

50

Security Filter Chain with HTTP Basic

- Features of the default configuration:

Function	Behaviour
URLs' protection	All requests require authentication (anyRequest().authenticated())
Login	No login page. Authentication is done via the Authorization header in HTTP requests
Logout	Not applicable (HTTP Basic is stateless)
Default user	Username: user Password: randomly generated at runtime (shown in console log)
PasswordEncoder	None or BCryptPasswordEncoder
Session Management	Each request must include authentication credentials ("no session")
CSRF protection	Disabled
UserDetailsService	InMemoryUserDetailsManager with the default user

51

What does the default configuration look like?

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults()) // <-- Abilita HTTP Basic
            .csrf(csrf -> csrf.disable()); // Di solito disabilitato per API REST
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user = User.withUsername("user")
            .password(passwordEncoder().encode("password"))
            .roles("USER")
            .build();
        return new InMemoryUserDetailsManager(user);
    }
}
```

When you define your own configuration, Spring disable the default one. You must manage everything yourself!

52

CSRF: enabled or disabled?

- CSRF protection ensures that state-changing operations (e.g. POST) can only be performed by requests originating from the legitimate user session
- How does it work:
 - We know that the first time a user connect to a web app they use a GET
 - When this happens, the web app generates a unique token (set in as a hidden field of a form in a HTML page, or in a header)
 - From now on, the web app accepts only requests for state-changing operations (e.g. POST) that contain this unique value in the header. The server verifies that the CSRF token matches the one associated with the user's session, ensuring that the request was intentionally made by the authenticated user and not forged by a third-party site.

53

CSRF: enabled or disabled?

- We can disabled CSRF protection in a couple of ways:

```
.csrf(csrf->csrf.disable())
or
http.csrf(AbstractHttpConfigurer::disable);
```

For the sake of simplicity, we will work with CSFR disabled

54

Using more than a chain

- To solve possible ambiguities due to headers, the best practice is to use two SecurityFilterChain:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // For API REST: HTTP Basic is adopted
    @Bean
    @Order(1)
    public SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
        http
            .securityMatcher("/api/**") // only /api/**
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults())
            .csrf(csrf -> csrf.disable());
        return http.build();
    }

    // For single-unit: form login is adopted
    @Bean
    public SecurityFilterChain webSecurity(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/login").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(Customizer.withDefaults());
        return http.build();
    }
}
```

It is used to establish an order of evaluation of the SecurityFilterChains

Use that only when you have more than one SecurityFilterChain. It just says: hey, use that part of config for URL starting with /api

55

What does happen at Sign-up?

- Spring Security does not implements Sign up automatically
- Thus:
 - You implement a registration form with the required fields
 - In your controller/service:
 - You hash the password using a PasswordEncoder
 - You save users to your data store (e.g., database) via a custom or standard UserDetailsManager
 - You may assign default roles/authorities (see later)

56

What does happens to requests after login?

- The authentication filter is automatically skipped because it is no longer necessary
- The Authorizaton filter (see next slides) is not skipped, because every request must be authorized even if the user is already logged in

57

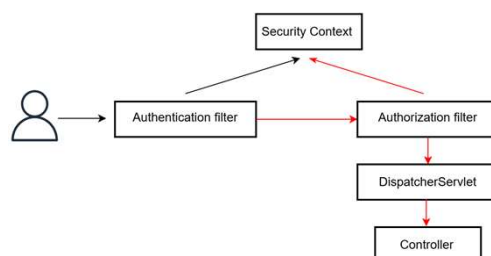
About custom login and logout

- If you want to customize the login form you have to:
 - Explicitly indicate in the security configuration the next pages in case login is successful or not
 - The action attribute of the form must be called /login
e.g. `th:action="@{/login}"`
- If you want to customize the logout form you have to:
 - Explicitly override the URL of Spring Security logout page and indicate the URL of your logout page

58

Spring Security's authorization architecture

- In most web app, once the application ends the authentication flow, it delegates the request to an authorization filter that allows / rejects the request based on the configured authorization rules
- The Authorization filter receives the request and asks the SecurityContext to obtain an Authentication object with the UserDetails (username, privileges and so on)

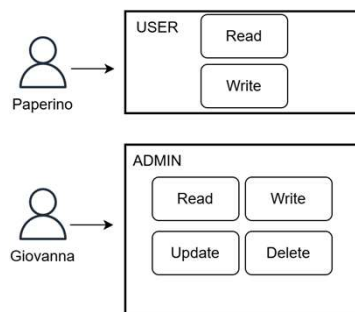


Based on the privileges users have, they can only execute a specific action

59

Authorities and roles

- Web apps provide privileges as authorities and roles
- Authorities are actions that a user can perform with a system resource
- Roles are another way to refer to what a user can do. They are specific combinations of authorities



We will just focus on roles

60

Authorities and roles

- Both authorities and roles are represented as Strings and can be wrapped in objects `SimpleGrantedAuthority` that implement the interface `GrantedAuthority`
- The `UserDetails` has a collection of `SimpleGrantedAuthority` instances. You can allow a user one or more privileges. The `getAuthorities()` method returns the collection of `GrantedAuthority` instances
- After successful authentication, the authorities become part of the details about the user that logged in

61

What about configuration?

- Example of configuration:

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .requestMatchers("/login", "/public/**", "/").permitAll()
        .anyRequest().permitAll()
    )
```

- Admin can login and have access to all the path /admin/** (e.g., admin/x; admin/x/y; admin/x/y/z) with only /* you can go down up to admin/y
- /login is accessible to all people
- All the other pages are public

62

Ok, so...

- In the configuration, to set constraints for user roles, you can use one of the following methods:
 - `hasRole()`—Receives as a parameter the role name for which the application authorizes the request
 - `hasAnyRole()`—Receives as parameters the role names for which the application approves the request
 - `anyRequest()` — Indicates that the rule applies to all the requests, regardless of the URL or HTTP method used. Invoking on that `.permitAll()` method allows access to all matched requests, authenticated or not, whereas `.authenticated` allow access to authenticated people only

`.anyRequest()` only at the end, and only if you want to define a “default” rule for everything else

63

.requestMatchers

- It is a method used to choose the requests to which we apply authorization configuration
- It is used inside `authorizeHttpRequests(...)`

Spring Security applies matchers top-down:
put more specific matchers first

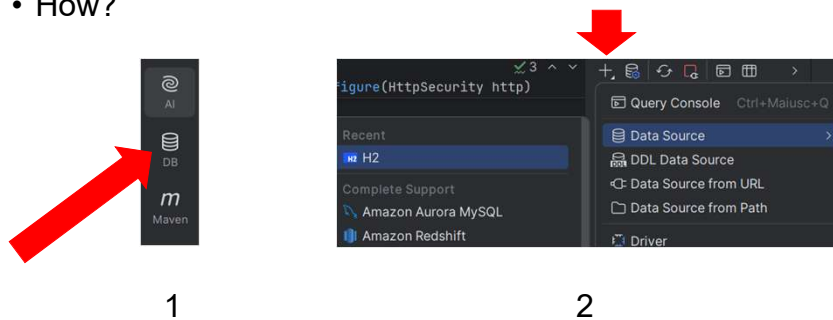
64

**esempio_security
(+OpenFeign0A)**

65

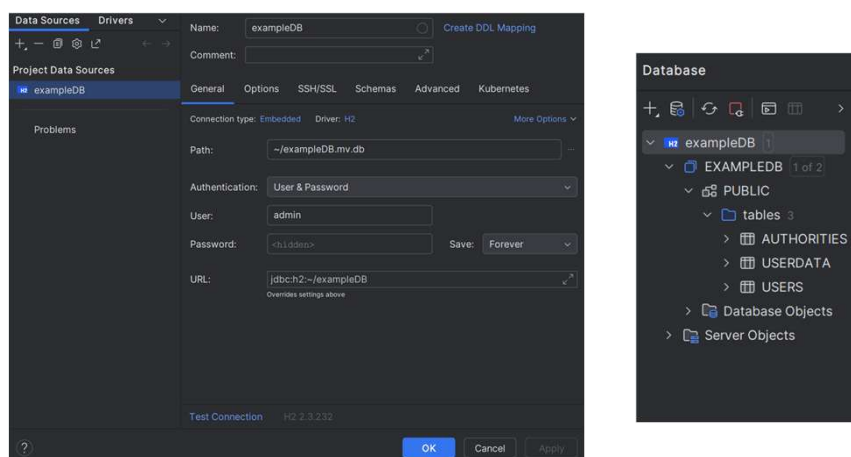
About H2 database

- We can access a H2 database directly in IntelliJ
- How?



66

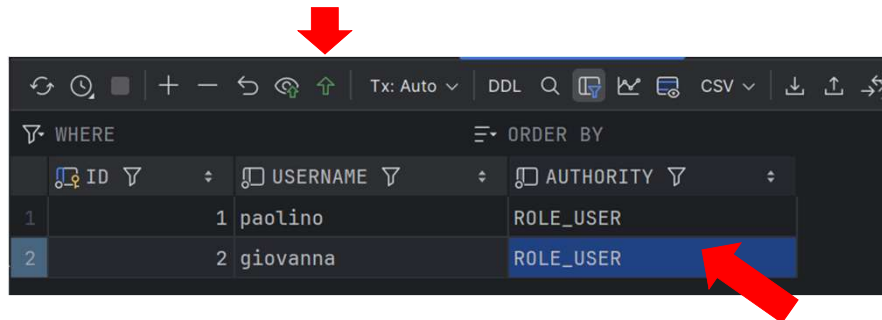
About H2 database



67

About H2 database

- If you want to create an admin: create a user and then modify their role from ROLE_USER to ROLE_ADMIN. After that click on the green-tinted arrow to validate the change



Remember: close the views of all the tables before to run the web app!!!!

68

More news on UserDetails

- Spring Security uses UserDetails object to manage user details
- Let's have a look at the methods of UserDetails

Method	Description
getAuthorities()	Returns the authorities granted to the user
getPassword()	Returns the password used to authenticate the user
getUsername()	Returns the username used to authenticate the user
isAccountNonExpired()	Indicates whether the user's account has expired
isAccountNonLocked()	Indicates whether the user is locked or unlocked
isEnabled()	Indicates whether the user is enabled or disabled
isCredentialsNonExpired()	Indicates whether the user's credentials (password) has expired

Remember that your User needs to be specialized to implement the UserDetails interface

69

About OpenFeign with Spring Security

- When you use OpenFeign with Spring Security you may experience conflicts in the order of the beans creation
- What we can do?
 - Just delay the creation of the part about OpenFeign
 - So:
 - We need to add an additional file, a service, that calls the OpenFeign methods
 - When you inject the OpenFeign proxy, take care to add `@Lazy` in the constructor